SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION **COURSE CODE: ITA0515** **Slot B**

COURSE OUTCOME COVERED
CO5: ***Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.*** **(BTL 6)**

Assessment Tool Weightage: 10%

QUESTIONS:4 **Total Marks 20** **Duration :60 Minutes**

Annexure A

Mock Interview

Code of Conduct:

I S.Jeswanth Kumar (192321056) ***certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.***

Signature of the student: S.Jeswanth Kumar

Rubrics for Evaluation

Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	***Marks(100)***
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

OpenCV Basics & Pipeline Thinking

1. Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.
2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.
3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.
4. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

ANSWERS

1. The image files are obtained from the specified folder using Python's glob or os module. Each image is loaded using cv2.imread(). The loaded image can then be resized using cv2.resize() to maintain a consistent resolution and reduce computational cost. If required, the image is converted from BGR to grayscale using cv2.cvtColor().

After preprocessing, the required computer vision operation such as filtering, edge detection, thresholding, or contour detection is performed. For example, cv2.Canny() can be used for edge detection. The processed image is then displayed using cv2.imshow() or stored using cv2.imwrite().

```
import cv2
import glob

image_paths = glob.glob("images/*.jpg")
for path in image_paths:
    image = cv2.imread(path)
    if image is None:
        continue
    image = cv2.resize(image, (640, 480))
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 100, 200)
    cv2.imshow("Processed Image", edges)
    if cv2.waitKey(1) & 0xFF == 27:
        break
cv2.destroyAllWindows()
```

The main challenges are different image resolutions, corrupted files, unsupported formats, large numbers of images, and memory consumption. These can be addressed by validating images before processing, resizing large images, processing them sequentially, avoiding unnecessary copies, and using parallel processing when the dataset is very large. This pipeline provides an efficient and organized method for processing multiple images.

2. When a video does not load properly in OpenCV, the problem can be caused by an incorrect file path, unsupported video format or codec, corrupted video, permission issues, or an incorrect OpenCV/FFmpeg configuration.

The first step is to verify that the video file exists and that the path is correct. The video should then be opened using cv2.VideoCapture() and its status should be checked using isOpened().

```
import cv2

cap = cv2.VideoCapture("video.mp4")
if not cap.isOpened():
    print("Unable to open video")
```

```
ret, frame = cap.read()
if not ret:
    print("Unable to read video frame")
```

If the video opens but frames cannot be read, the codec or video file may be the problem. The video properties such as frame width, height, FPS, and frame count can also be checked using cap.get().

A structured debugging process is therefore:

1. Verify the file path and file existence.
2. Check whether VideoCapture opens successfully.
3. Check whether individual frames are returned.
4. Verify the video format and codec.
5. Check video properties such as FPS and resolution.
6. Test the program with a known working video.
7. Update or reinstall OpenCV/FFmpeg if required.
8. Check whether the video file itself is corrupted.

This systematic approach helps identify whether the problem originates from the file, path, codec, software configuration, or frame-reading process.

3. To overlay text and shapes on a live video feed, the system should continuously capture frames from the camera, draw the required elements on each frame, and display the modified frame.

The pipeline is:
Camera → Frame Capture → Text and Shape Overlay → Display → Repeat

The camera can be accessed using cv2.VideoCapture(0). Text can be added using cv2.putText(), while shapes can be drawn using functions such as cv2.rectangle(), cv2.circle(), and cv2.line().

```
import cv2
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    cv2.putText(
        frame,
        "Live Video",
        (30, 50),
        cv2.FONT_HERSHEY_SIMPLEX,
        1,
        (0, 255, 0),
        2
    )
    cv2.rectangle(
        frame,
        (100, 100),
        (400, 300),
        (255, 0, 0),
        2
    )
    cv2.circle(
        frame,
```

```
        (250, 200),
        50,
        (0, 0, 255),
        2
    )
    cv2.imshow("Live Feed", frame)
    if cv2.waitKey(1) & 0xFF == 27:
        break
cap.release()
cv2.destroyAllWindows()
```

The important OpenCV functions are VideoCapture() for acquiring video, putText() for text, rectangle(), circle(), and line() for shapes, imshow() for displaying frames, and waitKey() for controlling the loop.
For efficient operation, unnecessary processing should be avoided inside the loop. The frame resolution can be reduced when high resolution is not required, and only the necessary overlays should be drawn. Finally, the camera and OpenCV windows must be released properly to prevent resource leakage.

4. An image may appear too dark because of poor lighting, low exposure, insufficient pixel intensity, or unsuitable image-processing operations. The processing pipeline should therefore include brightness and contrast enhancement.

The modified pipeline is:
Input Image → Validation → Color Conversion → Contrast Enhancement → Brightness Adjustment → Output

First, the image should be checked to ensure that it has been loaded correctly.

```
import cv2
image = cv2.imread("dark.jpg")
if image is None:
    print("Image could not be loaded")
```

For a grayscale image, histogram equalization can be applied using cv2.equalizeHist(). For images with uneven illumination, CLAHE (Contrast Limited Adaptive Histogram Equalization) provides better local contrast enhancement.

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)
enhanced = clahe.apply(gray)
cv2.imshow("Enhanced Image", enhanced)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Histogram equalization improves visibility by redistributing pixel intensities across a wider range. CLAHE enhances local contrast in different regions of the image while limiting excessive enhancement and noise amplification.
If necessary, brightness can also be increased by adjusting pixel intensity or using suitable color-space transformations. Therefore, the enhanced pipeline produces a brighter and more visible image while preserving important details.